

PubSub 限流器系统设计：一次完整的推演记录

题源：Coupang Senior SWE 面经（Glassdoor）中的系统设计原题：*design a rate limiter for PubSub。*

题面：有一个 PubSub 系统，很多 publisher 向不同的 topic 发消息。需要一个限流器，防止某个 publisher 或某个 topic 打爆下游。

1. 澄清：限流点在哪里

题面说”防止打爆下游”，但没说下游是谁。链路上有三段，每段都可能需要限流：

限流点	保护对象	性质
publisher → topic	防单个 publisher 打爆 topic	系统内部背压
topic → subscriber	防 topic 打穿 subscriber（push 模式下需要）	系统内部背压
user → subscriber	防用户打爆 subscriber	对外配额控制

两个观察值得说出口：

- 如果 subscriber 是拉模式，第二个限流点不需要 —— 消费速率由 subscriber 自己控制。这是 pull 相对 push 的天然优势。
- 第三个限流点和前两个性质不同 —— 前两个是内部背压，配额来自下游的实际处理能力；第三个是对外配额，数值由商业策略（计费、防滥用）决定。

本文只做第一段：publisher → topic。

2. 必须先定的三个 SLO

这三个不是形式，它们各自决定一个架构选择。

2.1 判定延迟

每条消息都要经过限流器做一次判定。这次判定本身花多少时间？

- 若要求**亚毫秒** → 必须本地内存判定，不能查远程存储
- 若**几毫秒可接受** → 可以用中心化计数服务

这是限流器自身的性能承诺，不是”限流前后的流量差”（那是效果指标，属于监控）。

本设计取**亚毫秒**，因此选**本地判定**。

2.2 精度：硬上限还是软上限

配额 1 万，允许短时冲到 1.1 万吗？

- 硬上限** → 需要全局强一致计数，要么每次查中心（慢），要么用复杂的分布式协议
- 软上限** → 本地计数 + 定期同步即可，快，但会超一点

关键判断：留 buffer 就等价于接受软上限。若下游真实上限是 2 万，把限流阈值设在 1 万，那么短时超过 1 万完全无害——真正的天花板还在很远。这个 buffer 的存在，让昂贵的全局强一致计数变得没有必要。

2.3 粒度：规则挂在哪个键上

粒度	计数器数量（1 万 topic × 10 万 publisher）
按 topic	1 万
按 publisher	10 万
按 (publisher, topic) 组合	理论上 10 亿

选**组合粒度**，因为它能定位到”具体是哪个 publisher 在打爆哪个 topic”，而不是一刀切。

10 亿这个数是理论上限。实际极度稀疏——多数 publisher 只发给少数几个 topic，活跃组合可能只有几十万到几百万。工程上：

- 计数器**按需创建**，不预分配
- 加 **TTL**，长期不活跃自动清理
- 配额也按需——绝大多数组合用默认值，只有少数需要单独配置

3. 规模假设

项	值
topic 数	1 万
publisher 数	10 万
topic 层集群	500 台
单机处理能力	2 万 msg/s
集群上限	1000 万 msg/s
单 topic 默认配额	1 万 msg/s（大 topic 可配至 50 万）
峰值总流量	500 万 msg/s

4. 部署形态：限流器与 topic 分片同机

大 topic（配额 50 万/秒）：分 25 台，每台承担 2 万配额，限流器与 topic 分片部署在同一台机器上。

小 topic：多个 topic 合并到一台机器（如 10 个/台），单台限流器全权负责该 topic 的判定。

为什么小 topic 不分片——见第 6 节的统计偏差分析。

合并部署的副作用：同机 topic 之间需要隔离。一个 topic 突发会吃掉整机容量、饿死同机的另外 9 个。除各自的配额外，还需要一条”单 topic 最多占整机 X% “的上限。

配额分配通常是混合策略：大客户按合同配置，其余按默认值或平均分。

5. 限流算法：滑动窗口

限流器的状态是（pub_id, 窗口数据），读写 O(1)，本地内存，判定 < **0.1ms**。

为什么不用固定窗口

固定窗口只需记（窗口开始时间，计数），实现最简单，但有**窗口边界的 2 倍突发**问题：

配额 1 万/秒，窗口按整秒切：

- 第 1 秒最后 100ms：发 1 万条（占满第 1 秒配额）
- 第 2 秒最前 100ms：窗口重置，又发 1 万条

→ 这 200ms 内实际通过 2 万条，是配额的 2 倍

限流器认为两个窗口都合规，但下游看到的是瞬时 2 倍冲击。

滑动窗口（双队列实现） 看的是”过去 1 秒”而非”当前这个整秒”，任意时间窗口内都不会出现 2 倍突发。

顺带解决可观测性

滑动窗口的队列可以顺便保留 **10 秒的 buffer**，记录被拒消息的 `pub_id / message_id`。2 万/秒 × 10 秒 = 20 万条记录，内存开销可忽略，但排查问题时价值极高。**同一个数据结构同时解决限流和可观测性。**

6. 路由：随机 vs 哈希

大 topic 分 25 台之后，publisher 的消息如何路由？

6.1 两种选择的代价

路由方式	组合配额的准确性	热点风险
按 <code>pub_id</code> 哈希	精确——同一 publisher 固定落一台，单机就能判准，无需跨机同步	重 publisher 打爆它所在的那台
随机 / 客户端选择	不准——流量散在 25 台，每台只看到 1/25	天然分散，无热点

哈希路由的热点问题很实在：若某个 publisher 占该 topic 流量的 50%，哈希后它全部落在一台上，那台承担 50% 而公平份额只有 4%。

6.2 随机路由下的统计偏差

选随机路由时，“各限流器负载差不多”这个假设**只在大流量下成立**。按二项分布计算：

topic 速率	每限流器均值	变异系数	P99 偏离均值
50 万/秒	20,000	0.69%	+1.6%
1 万/秒	400	4.90%	+11.4%
1000/秒	40	15.49%	+36%

大 topic 确实很均匀（2% 以内），小 topic 的相对偏差大得多。1000/秒 的 topic 若静态平均分配额，某台限流器可能比均值高 36%，导致误杀。

因此规则很干净：小 topic 不分片，单台全权负责；只有大 topic 才分片，而分片之后偏差自然就小了。

6.3 分层判定：本设计的选择

选随机路由，同时用分层判定解决组合配额判不准的问题：

路径	判定内容	位置	延迟
快路径	topic 总配额	本地内存	< 0.1ms
慢路径	publisher 级滥用	各限流器上报总控，总控下发封禁/降配指令	秒级

理由： 滥用检测本来就不需要毫秒级响应。快路径保证吞吐与延迟，慢路径牺牲实时性换准确性。

7. 客户端选路与退避

7.1 负载图与羊群效应

publisher 发送前读一张” 哪台空闲” 的表。两个问题：

表本身会成为热点 —— 每条消息发送前读一次，500 万 msg/s 就是 500 万次读/秒。解决：客户端**每分钟拉取一次**缓存到本地。

羊群效应 —— 缓存反而加剧了它：所有 publisher 在这一分钟内看到**同一张图**，若图上第 7 台最空，整整一分钟大家都往第 7 台发，把它打爆，下一分钟又全涌向别处，形成震荡。

对策：不选最空的，在较空的几台里随机挑一个（power of two choices）。一行逻辑就把羊群打散，而且缓存一分钟也无所谓了。

7.2 被限流后的退避

被拒后立即换一台重试，会导致**重试放大**：若 topic 整体过载，一条消息要试遍 25 台才放弃，过载时的重试流量是正常流量的 25 倍，让过载雪上加霜。

对策：指数退避 + 次数上限。等 2 秒、4 秒，最多三次即放弃。服务端在拒绝时可带 Retry-After，或返回明确的” topic 整体过载” 信号让客户端停止重试。

8. 容量余量：为什么设在 50%

上限 2 万/台，限流阈值设 1 万，机器数翻倍。

这不是保守，是排队论的要求。M/M/1 的逗留时间服从指数分布， $T(x)/T_mean = -\ln(1-x)$ ，即 **P99 = 4.6 × 均值**；而均值 = 服务时间 / (1-ρ)：

利用率 ρ	P99 / 服务时间
0.5	9.2×
0.7	15×
0.9	46×

跑到 90% 利用率时，尾延迟是服务时间的 46 倍。**50% 利用率同时保护了突发流量和 P99**，一举两得。

9. 监控指标

各限流器定期上报总控，聚合后：

指标	用途
拒绝率 （按 topic、按 publisher 分）	持续偏高说明配额或容量不匹配，是扩容信号
限流器之间的偏差	突然变大说明流量不均或出现热点 publisher
被拒消息的分布	集中在少数 publisher → 滥用；普遍分布 → 容量不足。 两种情况的处理完全不同
各限流器聚合总量	超过阈值告警

第三个指标最有价值：它把” 该封禁某个 publisher” 和” 该加机器” 这两个截然不同的结论区分开。

10. 设计要点回顾

1. **先确认限流点在哪一段**。三段链路性质不同：内部背压的配额来自下游能力，对外配额来自商业策略。若 subscriber 是拉模式，其中一

段根本不需要限流。

2. **buffer 的存在决定了不需要强一致计数。** 上限 2 万、阈值 1 万，短时超一点无害。这一个判断就砍掉了整个分布式一致性计数的复杂度。
3. **粒度决定计数器规模。** 组合粒度理论上 10 亿，实际稀疏——按需创建 + TTL 即可，不要被理论上界吓退。
4. **随机路由的均匀性只在大流量下成立。** 1000/秒 的 topic 偏差可达 36%，因此小 topic 不分片。
5. **组合配额判不准时，分层解决。** 快路径本地判 topic 配额（亚毫秒），慢路径离线聚合判 publisher 滥用（秒级）——因为滥用检测本来就不需要毫秒级响应。
6. **负载图 + 缓存会加剧羊群效应。** 对策不是提高刷新频率，而是”在较空的几台里随机挑”。
7. **重试必须退避且有次数上限，** 否则过载时重试流量会放大到机器数倍。
8. **50% 利用率同时是突发余量和 P99 保障，** 有排队论依据，不是拍脑袋。